

Object-Oriented Python — Unit 1

Object-Oriented Python

Topic	Coverage
Introduction to OOP	Concepts, Real-world Mapping, Procedural vs OOP
Classes and Objects	Blueprints, Instances, Memory, Creation Syntax
Constructor Method	<code>__init__()</code> , Self Parameter, Initialization
Instance Variables & Methods	Per-object Data, Behavior Functions
Class Variables & Methods	Shared Data, <code>@classmethod</code> , <code>@staticmethod</code>
Encapsulation & Abstraction	Access Modifiers, Data Hiding, Abstract Classes

Table of Contents

Table of Contents	1
Introduction to Object-Oriented Programming	1
1.1 What is Object-Oriented Programming?	1
1.2 How OOP Models the Real World	1
1.3 Procedural Programming vs Object-Oriented Programming	1
Procedural Programming	1
Object-Oriented Programming	1
1.4 The Four Pillars of OOP	1
1.5 Why OOP Matters in Python	1
Classes and Objects	1
2.1 What is a Class?	1
2.2 What is an Object?	1
2.3 Class vs Object — The Key Differences	1
2.4 Multiple Objects from One Class	1
2.5 How Python Stores Objects in Memory	1
Creating a Class & Objects	1
3.1 The class Keyword	1
Class Naming Rules	1
3.2 Defining Attributes and Methods Inside a Class	1
3.3 The self Parameter — Explained Thoroughly	1
3.4 Accessing Attributes and Calling Methods	1
__init__() Constructor Method	1
4.1 What is a Constructor?	1
4.2 Syntax of __init__()	1
4.3 Default Parameter Values in __init__()	1
4.4 The __str__() Method	1
Instance Variables and Methods	1
5.1 What are Instance Variables?	1
5.2 Defining Instance Variables	1
5.3 What are Instance Methods?	1
Class Variables and Methods	1

6.1 Instance Variables vs Class Variables	1
6.2 Defining and Using Class Variables	1
6.3 Class Methods using @classmethod	1
6.4 Static Methods using @staticmethod	1
Encapsulation & Abstraction	1
7.1 What is Encapsulation?	1
7.2 Access Modifiers in Python	1
7.3 Getter and Setter Methods	1
7.4 What is Abstraction?	1
7.5 Abstract Classes in Python	1



CHAPTER 1

Introduction to Object-Oriented Programming

Understanding the OOP Philosophy

Topics Covered in This Chapter

- What is Object-Oriented Programming?
- How OOP Models the Real World
- Procedural Programming vs OOP
- The Four Pillars of OOP
- Why OOP Matters in Python

Introduction to Object-Oriented Programming

Programming is essentially about giving instructions to a computer to solve a problem. As programs grow larger and more complex, programmers need better ways to organize and manage their code. Object-Oriented Programming (OOP) is one of the most powerful and widely used approaches to achieve this.

In this chapter, we will explore what OOP is, how it models the real world, and why it is the preferred approach for building large-scale software. By the end, you will understand the philosophy behind OOP and be fully prepared to start writing object-oriented Python code.

1.1 What is Object-Oriented Programming?

OOP

Object-Oriented Programming is a programming paradigm — a way of thinking about and structuring code — where a program is built around objects. Each object combines related data (what it knows) and behaviour (what it can do) into a single, self-contained unit.

The word paradigm simply means an approach or style. Just as you can solve a maths problem using different methods, you can write programs in different styles. OOP is one such style — and an extremely popular one.

**Analogy**

Think of your smartphone.

Your phone has **data** — your contacts, your photos, your settings. It also has **behaviour** — you can call someone, take a photo, or change the ringtone. The phone bundles all of this together as one complete, self-contained device. In OOP, an **object** works exactly the same way.

1.2 How OOP Models the Real World

One of the most powerful features of OOP is that it mirrors the way we naturally think about things in the real world. Everything around us can be thought of as an object with properties (data) and actions (behaviour).

Consider these real-world examples:

Real-World Object	Properties (Data)	Behaviour (Actions)
Bank Account	Account number, Holder name, Balance	Deposit money, Withdraw money, Check balance
Student	Name, Roll Number, Marks, Grade	Study, Appear for exam, Get result
Car	Brand, Model, Colour, Speed	Start engine, Accelerate, Brake, Stop
Book	Title, Author, ISBN, Pages	Open, Read, Bookmark, Close
Employee	Name, ID, Department, Salary	Work, Get promoted, Request leave, Resign

In OOP, we create these exact same structures in code — objects that hold related data and can perform specific actions. This makes programs much easier to understand because the code mirrors the way we already think about problems.

1.3 Procedural Programming vs Object-Oriented Programming

Before OOP, most programs were written using a procedural approach. It is important to understand both styles so you can appreciate why OOP was developed and when each is appropriate.

Procedural Programming

In procedural programming, you write a list of instructions, one after another, and the computer executes them in order. Functions are used to group instructions, but data and functions are kept separate.

Example — Managing a Bank Account Procedurally:

```
# Procedural approach
account_number = "ACC-001"
account_holder = "Rahul Sharma"
balance = 5000
```

```
def deposit(amount):
    global balance
    balance = balance + amount
```

```
def withdraw(amount):
    global balance
    if amount <= balance:
        balance = balance - amount
```

```
deposit(1000)
print(balance) # Output: 6000
```

The data (account_number, balance) and the functions (deposit, withdraw) exist separately. There is nothing stopping another part of the program from accidentally modifying the balance directly, which can lead to bugs.

Object-Oriented Programming

In OOP, the same bank account is represented as an object that bundles the data and functions together. You will learn the full syntax later — for now, just observe the overall structure.

```
# Object-Oriented approach
class BankAccount:
    def __init__(self, number, holder, balance):
        self.account_number = number
        self.account_holder = holder
        self.balance = balance
```

```
def deposit(self, amount):
    self.balance = self.balance + amount
```

```
def withdraw(self, amount):
    if amount <= self.balance:
        self.balance = self.balance - amount
```

```
account = BankAccount("ACC-001", "Rahul Sharma", 5000)
account.deposit(1000)
print(account.balance) # Output: 6000
```

Now the data and functions are neatly organised inside the BankAccount entity. The balance is protected and only accessible through the defined methods.

Procedural Programming	Object-Oriented Programming
Data and functions are separate	Data and functions are bundled together in objects

Harder to manage as code grows larger	Scales well — large programs stay organised
Difficult to reuse code in other projects	Objects and classes are highly reusable
No natural model for real-world things	Directly mirrors real-world entities
Risk of data being modified unexpectedly	Data can be protected from outside interference
Good for simple, linear scripts	Preferred for large, complex applications

1.4 The Four Pillars of OOP

OOP is built on four fundamental concepts, often called the Four Pillars. You will study each in depth throughout this textbook. Here is a brief introduction to all four so you can see the complete picture.

Pillar	Simple Meaning	Covered In
Encapsulation	Bundling data and methods together; hiding internal details from the outside world	Chapter 7 of this unit
Abstraction	Showing only what is necessary; hiding complex implementation details	Chapter 7 of this unit
Inheritance	A new class can reuse and extend the features of an existing class	Unit 2
Polymorphism	The same method name can behave differently depending on the object	Unit 2



Note

Do not worry about understanding all four pillars right now. This chapter introduces you to the idea. Each pillar will be taught fully, with examples, in the chapters ahead.

1.5 Why OOP Matters in Python

Python is a multi-paradigm language — it supports both procedural and object-oriented styles. However, Python is built with OOP as a core part of its design.

In fact, everything in Python is already an object. The number 5 is an integer object. The text "Hello" is a string object. Even a function is an object. This means learning OOP in Python is not just about writing new types of programs — it is about truly understanding how Python itself works at a fundamental level.



Real World Example

Major Python frameworks and tools — including Django (web development), scikit-learn (machine learning), and Pygame (game development) — are entirely built on OOP. Understanding OOP unlocks the ability to use, extend, and contribute to these powerful systems.

CHAPTER 2

Classes and Objects

*Blueprints and Real Instances***Topics Covered in This Chapter**

- What is a Class? The Blueprint Concept
- What is an Object? Creating Instances
- Class vs Object — Key Differences
- Multiple Objects from One Class
- How Python Stores Objects in Memory

Classes and Objects

The two most fundamental building blocks of Object-Oriented Programming are classes and objects. Every OOP program you will ever write revolves around these two concepts. This chapter gives you a deep, thorough understanding of both — with extensive examples.

2.1 What is a Class?

Class

A class is a blueprint or template that defines the structure — what data an entity will hold and what actions it can perform. A class itself does not hold any actual data. It simply describes what an object of that type should look like.

**Analogy**

Architectural Blueprint Analogy: An architect draws a blueprint for a house. The blueprint specifies: 3 bedrooms, 2 bathrooms, a kitchen, total area 1,500 sq ft. The blueprint itself is not a house — nobody lives in a blueprint. But using that same blueprint, a builder can construct many individual houses. Each house built from the same blueprint is a separate, real, physical structure.

In Python, **a class is the blueprint**. The individual houses built from it are **objects**.

Let us look at a simple, clear example. Imagine we are building a system to manage student records.

Defining a Student class:

```
class Student:
```



```
pass # 'pass' means the class body is empty for now
```

This tells Python: "There is a category of things called Student." It defines the name and reserves it. The word `pass` is a placeholder that says "nothing is inside yet." We will fill it in shortly.

2.2 What is an Object?

Object

An object is a specific, concrete instance created from a class. It is a real entity in memory that holds actual values for the data defined by its class. Every object created from the same class follows the same structure but holds its own independent data.



Analogy

Employee ID Card Analogy: A company uses a standard ID card template — same layout, same fields (Name, ID Number, Department, Photo). Each employee receives their own card with their unique information. The template is the **class**. Each individual card issued is an **object**.

Creating an object (instance) from a class:

```
class Student:
    pass
```

```
# Creating two student objects from the same class
student1 = Student()
student2 = Student()
```

```
print(student1) # Output: <__main__.Student object at 0x...>
print(student2) # Output: <__main__.Student object at 0x...>
```

Python confirms that `student1` and `student2` are both `Student` objects but stored at different memory addresses. They are completely independent of each other.

2.3 Class vs Object — The Key Differences

Class	Object
A blueprint / template / design	A real, concrete instance built from the class
Exists once — defined only once in code	Can exist many times — create as many as needed
Does not occupy data memory by itself	Each object occupies its own memory space
Describes what data will be stored	Actually stores real data values

Describes what actions can be performed	Actually performs those actions on its own data
Example: The Student class	Example: student1 = Student(), student2 = Student()

2.4 Multiple Objects from One Class

One of the most powerful features of a class is that it can be used to create unlimited independent objects. Each object has its own copy of the data but shares the same set of rules and behaviours defined by the class.

Let us build a more complete example. We will create a class for Employee records and then create multiple employee objects.

```
class Employee:
    def __init__(self, name, department, salary):
        self.name = name
        self.department = department
        self.salary = salary
```

```
def display_details(self):
    print("Name : ", self.name)
    print("Department : ", self.department)
    print("Salary : Rs.", self.salary)
```

```
# Creating three independent employee objects
emp1 = Employee("Priya Mehta", "Finance", 55000)
emp2 = Employee("Arjun Kapoor", "Engineering", 72000)
emp3 = Employee("Sneha Reddy", "HR", 48000)
```

```
emp1.display_details()
emp2.display_details()
emp3.display_details()
```

Output:

```
Name : Priya Mehta
Department : Finance
Salary : Rs. 55000
---
Name : Arjun Kapoor
Department : Engineering
Salary : Rs. 72000
```

Each employee object is completely independent. Changing emp1's salary will not affect emp2 or emp3 in any way. They all share the same structure (class) but hold different data.

2.5 How Python Stores Objects in Memory

When you create an object with a statement like `emp1 = Employee(...)`, Python does two things:

1. It allocates a new area of memory to store the object's data.
2. The variable `emp1` becomes a reference — a label that points to that memory location.



Important

Reference Behaviour: If you write `emp4 = emp1`, you are NOT creating a new employee. You are creating a second label pointing to the SAME object in memory. Any change made through `emp4` will also be visible through `emp1`. To create an independent copy, you need to use special techniques (covered in advanced units).



CHAPTER 3

Creating a Class & Objects

*Syntax, Rules, and Hands-On Practice***Topics Covered in This Chapter**

- The class Keyword and Naming Rules
- Defining Attributes and Methods Inside a Class
- The self Parameter Explained
- Creating Objects — Step-by-Step Syntax
- Accessing Data and Calling Methods
- Multiple Complete Examples

Creating a Class & Objects

In the previous chapter, you understood the concept of classes and objects. Now we will focus on the exact Python syntax used to create them. By the end of this chapter, you will be confidently writing complete class definitions and creating objects from them.

3.1 The class Keyword

In Python, you define a class using the class keyword, followed by the class name, and a colon. Everything inside the class is indented (just like an if block or a function).

```
class ClassName:
    # Class body goes here
    # Attributes and methods are defined inside
```

Class Naming Rules

Rule	Correct	Incorrect
Must start with a letter or underscore	class Student:	class 1Student:
Only letters, digits, underscores allowed	class BankAccount:	class Bank-Account:
Use PascalCase (CapWords) by convention	class CustomerOrder:	class customer_order:
Cannot use Python reserved words	class MyClass:	class for: / class if:

**Pro Tip**

PascalCase means every word starts with a capital letter, with no spaces or underscores: StudentRecord, BankAccount, ProductCatalog. This is the standard Python convention for class names.

3.2 Defining Attributes and Methods Inside a Class

A class body typically contains two types of things:

- **Attributes** — Variables that store data related to the object.
- **Methods** — Functions that define what the object can do.

Both attributes and methods are defined inside the class block. Methods look exactly like regular Python functions — but they always take `self` as their first parameter (explained in Section 3.3).

Example — A Product class for an e-commerce system:

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name      # Attribute: product name
        self.price = price    # Attribute: product price
        self.stock = stock    # Attribute: units in stock

    def display_product(self): # Method
        print("Product:", self.name)
        print("Price : Rs.", self.price)
        print("Stock :", self.stock, "units")

    def apply_discount(self, percent): # Method with parameter
        discount = self.price * (percent / 100)
        self.price = self.price - discount
        print("New price after discount: Rs.", self.price)

# Creating objects
p1 = Product("Laptop", 75000, 10)
p1.display_product()
p1.apply_discount(10) # Apply 10% discount
```

Output:

```
Product: Laptop
Price : Rs. 75000
```

```
Stock : 10 units
New price after discount: Rs. 67500.0
```

3.3 The self Parameter — Explained Thoroughly

The self parameter is one of the most important and initially confusing parts of Python OOP. Let us break it down clearly.

self

self is a reference to the current object — the specific instance on which a method is being called. It allows the method to access and modify the object's own data.



Analogy

Identity Badge Analogy: Imagine you are in a large office with 100 employees. When a manager calls out "update your own timesheet", everyone knows they must update their own — not someone else's. The word "own" in that instruction is like self in Python. It tells the method: "operate on THIS specific object's data."

Why self is necessary:

```
class Counter:
    def __init__(self):
        self.count = 0    # This object's own count
```

```
    def increment(self):
        self.count = self.count + 1
```

```
    def get_count(self):
        return self.count
```

```
c1 = Counter()
c2 = Counter()
```

```
c1.increment()
c1.increment()
c2.increment()
```

```
print(c1.get_count()) # Output: 2
print(c2.get_count()) # Output: 1
```

Notice that `c1` and `c2` each have their own count. When `c1.increment()` is called, `self` inside the method refers to `c1` — so `c1`'s count is updated. When `c2.increment()` is called, `self` refers to `c2`. This is the entire purpose of `self`.

**Important**

Python Rule: `self` must always be the first parameter of every method inside a class. However, when you CALL the method, you do not pass `self` — Python passes it automatically.

3.4 Accessing Attributes and Calling Methods

Once an object is created, you interact with it using the dot (`.`) operator. This is called dot notation.

```
object_name.attribute_name    # Access an attribute
object_name.method_name()     # Call a method
object_name.method_name(argument) # Call a method with argument
```

Complete Example — Library Book System:

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages
        self.is_available = True # Default: book is available
```

```
def display_info(self):
    print("Title  :", self.title)
    print("Author  :", self.author)
    print("Pages   :", self.pages)
    print("Available:", self.is_available)
```

```
def issue_book(self):
    if self.is_available:
        self.is_available = False
        print(self.title, "has been issued.")
    else:
        print(self.title, "is not currently available.")
```

```
def return_book(self):
    self.is_available = True
    print(self.title, "has been returned.")
```

```
# Creating and using objects
```

```
book1 = Book("Clean Code", "Robert C. Martin", 431)
book2 = Book("Python Crash Course", "Eric Matthes", 544)
```

```
book1.display_info()
book1.issue_book()    # Issue the book
book1.issue_book()    # Try to issue again — already out
book1.return_book()   # Return it
book1.issue_book()    # Issue again — now available
```

CHAPTER 4

__init__() Constructor Method

Initialising Objects the Right Way

Topics Covered in This Chapter

- What is a Constructor?
- The __init__() Method Explained
- Parameters in __init__()
- Default Parameter Values
- The __str__() Method for Clean Output
- Real-World Constructor Examples

`__init__()` Constructor Method

Every time you create an object, Python needs a way to set it up — to give it its initial state. This is the job of the constructor. In Python, the constructor is a special method named `__init__()`. It is one of the most important methods you will write in OOP.

4.1 What is a Constructor?

Constructor

A constructor is a special method that is automatically called by Python when a new object is created from a class. Its purpose is to initialise the object — to set up the object's attributes with starting values the moment the object comes into existence.



Analogy

New Employee Joining Analogy: When a new employee joins a company, HR immediately fills in their onboarding form — name, employee ID, department, starting salary. This is done once, at the very beginning. The `__init__()` method works the same way — it runs automatically the moment an object is created, setting up all its initial information.

4.2 Syntax of `__init__()`

The `__init__()` method has a specific syntax. The double underscores on both sides (`__`) indicate it is a special Python method — also called a dunder (double underscore) method.

```
class ClassName:
    def __init__(self, parameter1, parameter2, ...):
        self.attribute1 = parameter1
        self.attribute2 = parameter2
```



Note

The `__init__` method name must be spelled exactly: two underscores, then **init**, then two underscores. Python will NOT automatically call it if the spelling is wrong.

Example — Hospital Patient Registration:

```
class Patient:
    def __init__(self, patient_id, name, age, condition):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.condition = condition
        self.admitted = True # Automatic default value
```

```
def patient_summary(self):
```

```
print("Patient ID :", self.patient_id)
print("Name      :", self.name)
print("Age       :", self.age)
print("Condition :", self.condition)
print("Admitted  :", self.admitted)
```

```
p1 = Patient("P-2041", "Meera Joshi", 34, "Fever")
p2 = Patient("P-2042", "Ravi Kumar", 56, "Diabetes")
```

```
p1.patient_summary()
```

4.3 Default Parameter Values in __init__()

Sometimes, you want to create an object without providing every piece of information upfront. Python allows you to specify default values for parameters in `__init__()`. If the caller does not provide that value, the default is used.

```
class Subscription:
    def __init__(self, user_name, plan="Basic", duration=30):
        self.user_name = user_name
        self.plan      = plan
        self.duration  = duration # duration in days

    def display(self):
        print(self.user_name, "| Plan:", self.plan, "| Days:", self.duration)
```

```
# Uses all default values for plan and duration
s1 = Subscription("Alice")
# Overrides the default plan only
s2 = Subscription("Bob", "Premium")
# Provides all values explicitly
s3 = Subscription("Carol", "Enterprise", 365)
```

```
s1.display() # Alice | Plan: Basic | Days: 30
s2.display() # Bob   | Plan: Premium | Days: 30
s3.display() # Carol | Plan: Enterprise | Days: 365
```

4.4 The __str__() Method

When you print an object directly, Python shows something like `<__main__.Student object at 0x...>`. This is not very readable. The `__str__()` method allows you to define a human-friendly text representation of your object.

```
class Invoice:
    def __init__(self, invoice_no, client, amount):
        self.invoice_no = invoice_no
        self.client = client
        self.amount = amount
```

```
def __str__(self):
    return f"Invoice {self.invoice_no} | Client: {self.client} | Amount: Rs.{self.amount}"
```

```
inv1 = Invoice("INV-001", "TechCorp Ltd.", 85000)
print(inv1) # Invoice INV-001 | Client: TechCorp Ltd. | Amount: Rs.85000
```

Now printing the object gives a clean, informative output instead of a memory address. The `__str__()` method is called automatically whenever you use `print()` on an object.



CHAPTER 5

Instance Variables and Methods

*Per-Object Data and Behaviour***Topics Covered in This Chapter**

- What are Instance Variables?
- Defining Instance Variables in `__init__()`
- Modifying Instance Variables
- What are Instance Methods?
- Instance Methods with Parameters
- Multiple Methods Working Together

Instance Variables and Methods

Now that you understand classes, objects, and the constructor, it is time to go deeper into the two core elements that make every object unique: instance variables (which store the object's own data) and instance methods (which define what the object can do with that data).

5.1 What are Instance Variables?

Instance Variable

An instance variable is a variable that belongs to a specific object (instance) of a class. Each object has its own copy of the instance variables, and changes to one object's variables do not affect any other object.

**Analogy**

Passport Analogy: Every Indian citizen has a passport. The passport has the same structure: name, date of birth, passport number, expiry date. But each person's passport holds completely different values. Changing the expiry date on your passport does not change it on anyone else's. Each passport holds its own independent data — just like instance variables.

5.2 Defining Instance Variables

Instance variables are defined inside `__init__()` using the `self.variable_name = value` syntax. Every attribute assigned to `self` becomes an instance variable.

```
class Vehicle:
```

```
def __init__(self, brand, model, year, fuel_type):
    self.brand = brand # instance variable
    self.model = model # instance variable
    self.year = year # instance variable
    self.fuel_type = fuel_type # instance variable
    self.speed = 0 # starts at 0 for every new vehicle
```

```
v1 = Vehicle("Maruti", "Swift", 2022, "Petrol")
v2 = Vehicle("Tata", "Nexon", 2023, "Electric")
v3 = Vehicle("Honda", "Activa", 2021, "Petrol")
```

```
# Each object has independent data
print(v1.brand, v1.model) # Maruti Swift
print(v2.brand, v2.model) # Tata Nexon
v1.speed = 60 # Only v1's speed changes
print(v1.speed) # 60
print(v2.speed) # 0 — completely unaffected
```

5.3 What are Instance Methods?

Instance Method

An instance method is a function defined inside a class that operates on the data of a specific object. It always takes `self` as its first parameter, giving it access to that object's instance variables.

Example — Online Order System:

```
class Order:
    def __init__(self, order_id, customer, total):
        self.order_id = order_id
        self.customer = customer
        self.total = total
        self.status = "Pending"
```

```
def confirm_order(self): # instance method
    self.status = "Confirmed"
    print(f"Order {self.order_id} confirmed for {self.customer}.")
```

```
def apply_coupon(self, discount): # instance method with parameter
    self.total = self.total - discount
    print(f"Discount applied. New total: Rs.{self.total}")
```

```
def cancel_order(self):
    if self.status == "Pending":
        self.status = "Cancelled"
        print(f"Order {self.order_id} cancelled.")
    else:
        print("Cannot cancel — order is already", self.status)
```

```
def order_summary(self):
    print(f"Order: {self.order_id} | Customer: {self.customer}")
    print(f"Total: Rs.{self.total} | Status: {self.status}")
```

```
o1 = Order("ORD-501", "Kavya Singh", 3500)
o1.order_summary()
o1.confirm_order()
o1.apply_coupon(500)
o1.cancel_order() # Cannot cancel — already confirmed
o1.order_summary()
```

Output:

```
Order: ORD-501 | Customer: Kavya Singh
Total: Rs.3500 | Status: Pending
Order ORD-501 confirmed for Kavya Singh.
Discount applied. New total: Rs.3000
Cannot cancel — order is already Confirmed
Order: ORD-501 | Customer: Kavya Singh
Total: Rs.3000 | Status: Confirmed
```

**Pro Tip**

Instance methods should do one thing well. Keep each method focused on a single responsibility — this makes your code easy to read, test, and maintain. This principle is known as the Single Responsibility Principle.

CHAPTER 6

Class Variables and Methods

Shared Data Across All Objects

Topics Covered in This Chapter

- Instance Variables vs Class Variables
- Defining Class Variables
- Accessing Class Variables
- Class Methods with @classmethod
- Static Methods with @staticmethod
- When to Use Each Type

Class Variables and Methods

So far, every variable and method we have worked with has been instance-specific — each object gets its own copy. But sometimes, you need data or behaviour that is shared across all objects of a class. This is where class variables and class methods come in.

6.1 Instance Variables vs Class Variables

Instance Variable	Class Variable
Defined inside <code>__init__()</code> using <code>self</code>	Defined directly inside the class, outside any method
Each object has its own independent copy	All objects share a single copy of this variable
Accessed via: <code>self.variable_name</code>	Accessed via: <code>ClassName.variable_name</code>
Change to one object does not affect others	Changing it via the class name affects all objects
Example: employee's own salary	Example: company name shared by all employees

6.2 Defining and Using Class Variables



Analogy

University Notice Board Analogy: Every student in a university has their own personal details — roll number, name, marks. But the university name, the current academic year, and the passing percentage are the same for every student. These shared values are like class variables — they belong to the class (the university) rather than any individual object (student).

Example — University Student System:

```
class Student:
    university = "National Institute of Technology" # Class variable
    passing_marks = 40 # Class variable
    total_students = 0 # Class variable: counter
```

```
def __init__(self, name, roll_no, marks):
    self.name = name # Instance variable
    self.roll_no = roll_no # Instance variable
    self.marks = marks # Instance variable
    Student.total_students += 1 # Shared counter increments
```

```
def result(self):
    if self.marks >= Student.passing_marks:
        print(f"{self.name}: PASS")
    else:
        print(f"{self.name}: FAIL")
```

```
s1 = Student("Ananya", "NIT-001", 78)
s2 = Student("Dev", "NIT-002", 35)
s3 = Student("Lena", "NIT-003", 55)
```

```
s1.result() # Ananya: PASS
s2.result() # Dev: FAIL
s3.result() # Lena: PASS
```

```
# Access class variable via class name or any object
print(Student.university) # National Institute of Technology
print(Student.total_students) # 3
print(s1.university) # Also works: National Institute of Technology
```

6.3 Class Methods using @classmethod

A class method is a method that operates on the class itself rather than on individual objects. It is defined using the `@classmethod` decorator and takes `cls` (the class) as its first parameter instead of `self`.

@classmethod

A decorator that marks a method as belonging to the class rather than to any specific instance. It receives the class itself as its first parameter (`cls`), and can read or modify class-level data.

```
class Company:
```



```
company_name = "InnoTech Solutions"
employee_count = 0
```

```
def __init__(self, name, role):
    self.name = name
    self.role = role
    Company.employee_count += 1
```

```
@classmethod
def get_company_info(cls):    # cls = the Company class
    print(f"Company: {cls.company_name}")
    print(f"Total Employees: {cls.employee_count}")
```

```
@classmethod
def change_company_name(cls, new_name):
    cls.company_name = new_name
```

```
e1 = Company("Pooja", "Designer")
e2 = Company("Harsh", "Developer")
```

```
Company.get_company_info()
# Output: Company: InnoTech Solutions | Total Employees: 2
```

```
Company.change_company_name("NovaTech Global")
Company.get_company_info()
# Output: Company: NovaTech Global | Total Employees: 2
```

6.4 Static Methods using @staticmethod

A static method is a method that does not depend on either the object (self) or the class (cls). It is a utility function that belongs inside the class because it is logically related to it, but does not need access to any class or instance data.

```
class MathHelper:
    @staticmethod
    def square(number):
        return number * number
```

```
@staticmethod
def percentage(marks, total):
    return (marks / total) * 100
```

```
@staticmethod
def is_even(n):
    return n % 2 == 0
```

```
# Called on class directly — no object needed
print(MathHelper.square(9))      # 81
print(MathHelper.percentage(85, 100)) # 85.0
print(MathHelper.is_even(14))    # True
```

Method Type	First Parameter	Can Access	Decorator	Call Style
Instance Method	self (the object)	Instance variables, class variables	None needed	object.method()
Class Method	cls (the class)	Class variables only	@classmethod	ClassName.method() or object.method()
Static Method	None (no self or cls)	Neither — pure utility	@staticmethod	ClassName.method()

ETIHANS
CLOUD & AI ACADEMY



CHAPTER 7

Encapsulation & Abstraction

Protecting Data and Simplifying Complexity

Topics Covered in This Chapter

- What is Encapsulation?
- Public, Protected, and Private Attributes
- Getter and Setter Methods
- The @property Decorator
- What is Abstraction?
- Abstract Classes with abc Module

Encapsulation & Abstraction

Two of the four pillars of OOP — Encapsulation and Abstraction — both deal with how information is hidden and organized in a program. While they are related concepts, they solve different problems. This chapter covers both in depth.

7.1 What is Encapsulation?

Encapsulation

Encapsulation is the principle of bundling an object's data (attributes) and the methods that operate on that data together in one unit (the class), while restricting direct access to some of the object's internal details from the outside.

**Analogy**

ATM Machine Analogy: When you use an ATM, you insert your card, enter your PIN, and select how much to withdraw. You do not touch the cash storage mechanism, the database connection, or the network transaction system. All of that is hidden from you. You only interact through the provided interface (the buttons and screen). The internal workings are encapsulated — protected from direct access. This is exactly how encapsulation works in OOP.

7.2 Access Modifiers in Python

Python uses a naming convention (not strict enforcement like Java or C++) to indicate how accessible a variable or method should be.

Type	Syntax	Meaning
Public	<code>self.balance</code>	Accessible from anywhere — inside and outside the class
Protected	<code>self._balance</code>	Single underscore: intended for internal/subclass use only. A convention — Python does not enforce this.
Private	<code>self.__balance</code>	Double underscore: Python applies name mangling, making it harder (but not impossible) to access from outside.

Example — Bank Account with Encapsulation:

```
class BankAccount:
    def __init__(self, account_no, holder, balance):
        self.account_no = account_no # public
        self._holder = holder # protected
        self.__balance = balance # private
```

```
    def deposit(self, amount): # public method
        if amount > 0:
            self.__balance += amount
            print(f"Deposited Rs.{amount}. Balance: Rs.{self.__balance}")
```

```
    def withdraw(self, amount):
        if amount > self.__balance:
            print("Insufficient balance.")
        elif amount <= 0:
            print("Invalid amount.")
        else:
            self.__balance -= amount
            print(f"Withdrawn Rs.{amount}. Balance: Rs.{self.__balance}")
```

```
    def get_balance(self): # controlled read access
        return self.__balance
```

```
acc = BankAccount("ACC-901", "Nisha Patel", 10000)
acc.deposit(5000) # Rs.5000 deposited. Balance: Rs.15000
acc.withdraw(3000) # Rs.3000 withdrawn. Balance: Rs.12000
print(acc.get_balance()) # 12000
```

```
# Direct access attempt — Python makes this difficult:
```

```
# print(acc.__balance) # AttributeError!
```

**Important**

Name Mangling: When you use double underscore (`__`), Python internally renames the attribute to `__ClassName__attributeName`. So `self.__balance` in `BankAccount` becomes `__BankAccount__balance`. While technically still accessible, this strongly discourages direct access and prevents accidental name collisions in subclasses.

7.3 Getter and Setter Methods

Since private attributes cannot be accessed directly from outside, we provide controlled access through getter and setter methods. This is a key pattern in encapsulation.

Getter — a method that returns the value of a private attribute.

Setter — a method that allows setting a new value for a private attribute, with built-in validation.

```
class Employee:
    def __init__(self, name, salary):
        self.__name = name
        self.__salary = salary
```

```
# Getter for salary
def get_salary(self):
    return self.__salary
```

```
# Setter for salary — with validation
def set_salary(self, new_salary):
    if new_salary < 0:
        print("Error: Salary cannot be negative.")
    elif new_salary < 10000:
        print("Error: Salary below minimum wage.")
    else:
        self.__salary = new_salary
        print(f"Salary updated to Rs.{self.__salary}")
```

```
emp = Employee("Rohan Das", 45000)
print(emp.get_salary())    # 45000
emp.set_salary(55000)      # Salary updated to Rs.55000
emp.set_salary(-500)       # Error: Salary cannot be negative.
```

7.4 What is Abstraction?

Abstraction

Abstraction means hiding the complex internal implementation details of an object and exposing only what is necessary for the user to interact with. It lets the programmer work with objects at a high level without needing to know how they work internally.



Analogy

Car Dashboard Analogy: When you drive a car, you use the steering wheel, accelerator, brake, and gear lever. You do not need to know how the combustion engine works, how the transmission system changes gears, or how the ABS brakes function. All of that complexity is hidden — abstracted away. You interact with a simple interface. Abstraction in OOP achieves exactly the same thing for code.

7.5 Abstract Classes in Python

Python provides abstract classes through the `abc` module (Abstract Base Classes). An abstract class is a class that cannot be instantiated directly — it is designed to be a parent template for other classes.

An abstract class can define abstract methods — methods that have no body and must be implemented by any subclass.

```
from abc import ABC, abstractmethod
```

```
class PaymentMethod(ABC): # Abstract base class
```

```
@abstractmethod
```

```
def process_payment(self, amount): # Abstract method — no body
    pass
```

```
@abstractmethod
```

```
def get_payment_details(self):
    pass
```

```
def show_receipt(self, amount): # Concrete method — has a body
    print(f"--- Receipt ---")
    print(f"Amount paid: Rs.{amount}")
    print(f"Thank you for your payment.")
```

```
# Concrete subclass — MUST implement all abstract methods
```

```
class CreditCard(PaymentMethod):
    def __init__(self, card_no, holder):
```

```
self.__card_no = card_no
self.holder = holder
```

```
def process_payment(self, amount): # Implementing abstract method
    print(f"Processing Rs.{amount} via Credit Card.")
```

```
def get_payment_details(self): # Implementing abstract method
    print(f"Credit Card: XXXX-XXXX-XXXX-{self.__card_no[-4:]}")
```

```
class UPIPayment(PaymentMethod):
    def __init__(self, upi_id):
        self.upi_id = upi_id
```

```
def process_payment(self, amount):
    print(f"Processing Rs.{amount} via UPI: {self.upi_id}")
```

```
def get_payment_details(self):
    print(f"UPI ID: {self.upi_id}")
```

```
# Using the concrete classes
c = CreditCard("1234567890123456", "Aryan Malhotra")
c.get_payment_details()
c.process_payment(2500)
c.show_receipt(2500)
```

```
u = UPIPayment("aryan@paytm")
u.process_payment(1200)
u.show_receipt(1200)
```

```
# Cannot instantiate an abstract class directly:
# p = PaymentMethod() # TypeError!
```

Encapsulation	Abstraction
Focuses on HOW data is protected	Focuses on WHAT details are hidden from users
Achieved by making attributes private and providing controlled methods	Achieved by defining abstract interfaces that hide implementation
Controls access to data	Controls visibility of complexity
Example: private <code>__balance</code> with deposit/withdraw methods	Example: <code>PaymentMethod</code> abstract class hides processing logic

Keeps data safe inside the class	Gives users a simple, clean interface to work with
----------------------------------	--



Real World Example

Where You See These Principles Daily:

Django ORM (Encapsulation): When you query a database with Django, the database connection, SQL query construction, and result parsing are all encapsulated inside model classes. You call `order.save()` and Django handles everything internally.

Python's file handling (Abstraction): When you call `open("file.txt")`, you do not need to know how the operating system manages file descriptors, buffers, or disk sectors. Python abstracts all of that away behind a simple, clean interface.

